Code-Layout, Readability and Reusability

Date	1 October 2026
Team ID	SWTID-2026-2430
Project Name	Team Enrollment Confirmation - FitBuddy – AI Fitness Plan Generator using Gemini Models
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the FitBuddy codebase in terms of structure, readability, maintainability, and reuse. The project uses FastAPI, Google Gemini models, SQLite with SQLAlchemy, HTML/CSS, and Jinja2. The work is divided among the five team members according to the given name order, covering model selection, AI/core functionality, backend routing and storage, frontend development, and deployment/testing.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs are used throughout the Python and HTML code.	Yes	Consistent formatting improves readability.
2	Proper File Structure	Files and folders are logically organized into app, templates and static resources.	Yes	Modular project structure is maintained.
3	Meaningful Variable Names	Variables and parameters clearly represent user data, goals, intensity and plans.	Yes	Names are descriptive and easy to understand.
4	Function / Method Names	Functions such as generate_workout_gemini() and update_workout_plan() describe their purpose.	Yes	Function names support quick understanding and reuse.
5	Code Comments	Important logic and AI/database operations are documented where required.	Yes	Comments/documentation explain major modules and flow.
6	Modular Design	AI generation, database access, routing and templates are separated into modules.	Yes	Modules can be maintained and reused independently.
7	No Redundant Code	Common operations are kept in reusable functions instead of repeated blocks.	Yes	Reusable functions reduce duplication.
8	Error Handling	Input validation and application-level handling are included around user and API processing.	Partial	Further exception handling can be added for production deployment.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Model Selection & Architecture – Divya.D	Python, FastAPI, Gemini API	Gemini integration and overall application flow	High
2	Core AI Functions – Deepika.M	Python, Gemini 1.5 Pro / Flash	Workout generation, nutrition tips and feedback updates	High
3	Backend & Database – Dharani.T	FastAPI, Pydantic, SQLAlchemy, SQLite	routes.py, user/plan storage and admin data access	High

4	Frontend & Templates – Divya shree.R	HTML, CSS, Jinja2	index.html, result.html and all_users.html	High
5	Deployment & Testing – DivyaDharshini.V	Uvicorn, Python, Git	Local deployment, API testing and verification	Medium

Team Work Allocation

S.No	Team Member	Assigned Work	Main Deliverables
1	Divya.D	Model selection and application architecture	Research Gemini models; define FastAPI + Gemini + SQLite + Jinja2 architecture; set up development environment.
2	Deepika.M	Core functionality and Gemini integration	Develop workout-plan generation, nutrition/recovery tip generation and feedback-based plan updating.
3	Dharani.T	Backend routing and database management	Develop routes.py logic, validate user input, connect routes to AI functions, and manage SQLite/SQLAlchemy storage.
4	Divya shree.R	Frontend UI and Jinja2 templates	Design responsive pages; develop index.html, result.html and all_users.html; bind backend data to templates.
5	DivyaDharshini.V	Deployment, testing and verification	Prepare local deployment, configure environment variables, run Uvicorn, test endpoints and verify application flow.

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clear separation of frontend, backend, AI generation and database responsibilities.
Readability	5	Descriptive functions, organized routes and structured templates make the code easy to follow.
Reusability	5	AI and database operations are implemented as reusable functions/modules.
Documentation / Comments	4	Project documentation explains the major activities and modules; more inline comments can be added where needed.
Overall Score	5	The project demonstrates a well-organized modular design suitable for further extension.

Project Summary:

FitBuddy is a web-based application that uses Google Gemini models to generate personalized 7-day workout plans and nutrition or recovery tips. The project uses FastAPI for backend logic, SQLAlchemy with SQLite for data storage, and Jinja2 templates for the frontend. Users can submit feedback to update an existing plan, while an admin view can display registered users and their original and updated plans. The modular structure supports maintenance, testing and future reuse.

Team Members	Project
Divya.D Deepika.M Dharani.T Divya shree.R DivyaDharshini.V	FitBuddy – AI Fitness Plan Generator using Gemini Models